Научно исследовательский университет ИТМО
Факультет систем управления и робототехники

Отчет по лабораторной работе №5
Кластеризация и PageRank

Выполнил студент группы R3280 Мовчан И. Е.
Преподаватель Перегудин А. А.

Санкт-Петербург
2024

1 Кластеризация

1.1 Выбор графа

Для начала выберем связаный граф, состоящий из 21 вершины. В нём каждая вершина представляет из себя человека, а ребро - отношение дружбы между соответствующими людьми (считаем, что если человек с кем-то дружит, то это происходит взаимно).

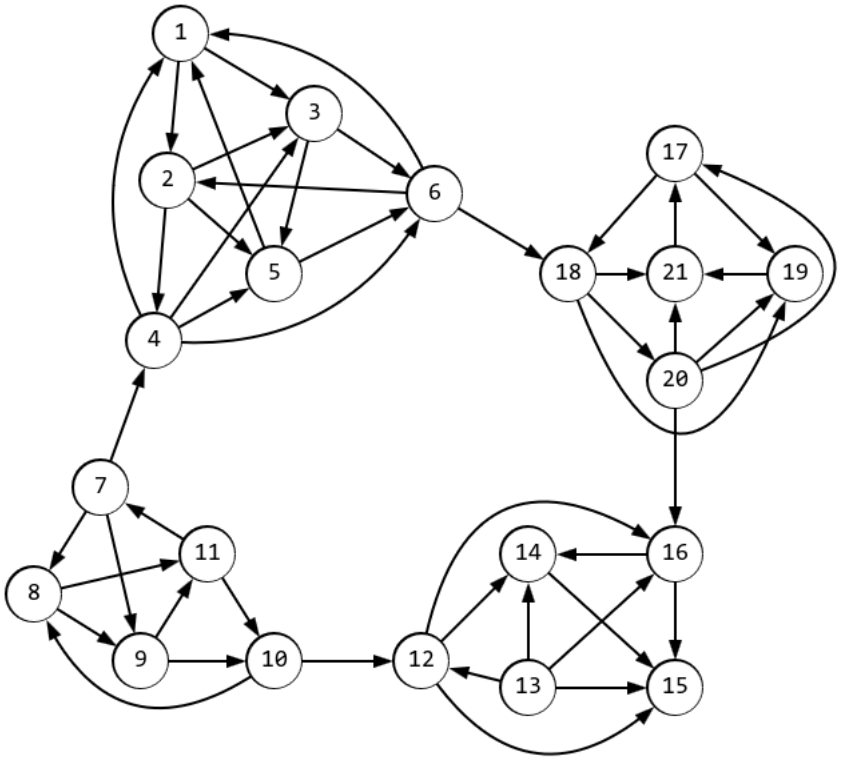


Рис. 1: Граф

Человеческий глаз запросто может выделить в нём 4 кластера (1-6, 7-10, 12-16, 17-21). Посмотрим, сможет ли это сделать компьютер? Для этого немного проанализируем наш граф.

1.2 Лапласиан, собственные числа и вектора

Итак, Лапласиан L нашего графа будет иметь следующий вид:

$$L = \begin{bmatrix}
5 & -1 & -1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & 5 & -1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & -1 & 5 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & -1 & -1 & 6 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & -1 & -1 & -1 & 5 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & -1 & -1 & -1 & -1 & 6 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\
0 & 0 & 0 & -1 & 0 & 0 & 4 & -1 & -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & 4 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 4 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 4 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 & 4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 5 & -1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 4 & -1 & -1 & -1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 4 & -1 & -1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 & 4 & -1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 & 5 & 0 & 0 & 0 & -1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & -1 & -1 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 5 & -1 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 4 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 & 5 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 & -1 & 4
\end{bmatrix}$$

Выглядит довольно-таки устрашающе, но это только на первый взгляд.

Собственные вектора будут иметь следующий вид:

$$v_1 = \begin{bmatrix}
0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22 \\ 0.22
\end{bmatrix}, v_2 = \begin{bmatrix}
-0.31 \\ -0.31 \\ -0.31 \\ -0.27 \\ -0.31 \\ -0.26 \\ -0.05 \\ 0.03 \\ 0.03 \\ 0.1 \\ 0.03 \\ 0.28 \\ 0.32 \\ 0.32 \\ 0.32 \\ 0.28 \\ 0.02 \\ -0.03 \\ 0.02 \\ 0.07 \\ 0.02
\end{bmatrix}, v_3 = \begin{bmatrix}
0.0 \\ 0.0 \\ 0.0 \\ 0.04 \\ 0.0 \\ -0.04 \\ 0.28 \\ 0.33 \\ 0.33 \\ 0.28 \\ 0.33 \\ 0.05 \\ -0.0 \\ -0.0 \\ -0.0 \\ -0.05 \\ -0.33 \\ -0.28 \\ -0.33 \\ -0.28 \\ -0.33
\end{bmatrix} v_4 = \begin{bmatrix}
-0.2 \\ -0.2 \\ -0.2 \\ -0.14 \\ -0.2 \\ -0.14 \\ 0.19 \\ 0.26 \\ 0.26 \\ 0.17 \\ 0.26 \\ -0.19 \\ -0.27 \\ -0.27 \\ -0.27 \\ -0.19 \\ 0.26 \\ 0.18 \\ 0.26 \\ 0.17 \\ 0.26
\end{bmatrix}, v_5 = \begin{bmatrix}
-0.06 \\ -0.06 \\ -0.06 \\ 0.15 \\ -0.06 \\ -0.05 \\ 0.65 \\ 0.01 \\ 0.01 \\ -0.67 \\ 0.01 \\ -0.24 \\ 0.1 \\ 0.1 \\ 0.1 \\ 0.07 \\ -0.0 \\ -0.02 \\ -0.0 \\ 0.03 \\ -0.0
\end{bmatrix}$$

$$v_6 = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ 0.08 \\ 0.18 \\ -0.0 \\ -0.27 \\ -0.0 \\ 0.64 \\ -0.65 \\ -0.12 \\ 0.13 \\ -0.14 \\ 0.0 \\ -0.01 \\ 0.13 \\ 0.01 \end{bmatrix}, v_7 = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ 0.09 \\ -0.26 \\ -0.0 \\ 0.17 \\ -0.0 \\ 0.37 \\ 0.1 \\ -0.3 \\ -0.17 \\ 0.63 \\ -0.0 \\ -0.44 \\ -0.17 \\ -0.02 \end{bmatrix}, v_8 = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ -0.01 \\ 0.03 \\ -0.0 \\ -0.02 \\ -0.0 \\ 0.05 \\ 0.07 \\ 0.05 \\ -0.17 \\ -0.31 \\ 0.0 \\ -0.36 \\ -0.17 \\ 0.84 \end{bmatrix}, v_9 = \begin{bmatrix} 0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ 0.42 \\ -0.02 \\ 0.0 \\ -0.4 \\ 0.0 \\ -0.32 \\ -0.25 \\ 0.54 \\ 0.03 \\ 0.31 \\ -0.0 \\ -0.33 \\ 0.03 \\ -0.01 \end{bmatrix}, v_{10} = \begin{bmatrix} 0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ -0.11 \\ -0.21 \\ -0.0 \\ 0.32 \\ -0.0 \\ -0.22 \\ -0.16 \\ -0.14 \\ 0.52 \\ -0.1 \\ 0.0 \\ -0.45 \\ 0.52 \\ 0.03 \end{bmatrix}$$

$$v_{11} = \begin{bmatrix} -0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ -0.64 \\ 0.3 \\ 0.0 \\ 0.34 \\ 0.0 \\ 0.01 \\ -0.31 \\ 0.43 \\ -0.13 \\ 0.24 \\ 0.0 \\ -0.1 \\ -0.13 \\ -0.01 \end{bmatrix}, v_{12} = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ -0.22 \\ 0.65 \\ 0.0 \\ -0.43 \\ -0.0 \\ -0.11 \\ 0.3 \\ -0.32 \\ 0.13 \\ 0.17 \\ -0.0 \\ -0.26 \\ 0.13 \\ -0.04 \end{bmatrix}, v_{13} = \begin{bmatrix} 0.06 \\ 0.06 \\ 0.06 \\ -0.26 \\ 0.06 \\ 0.08 \\ -0.53 \\ 0.28 \\ 0.28 \\ -0.36 \\ 0.28 \\ -0.44 \\ 0.13 \\ 0.13 \\ 0.13 \\ 0.04 \\ -0.05 \\ 0.1 \\ -0.05 \\ 0.05 \\ -0.05 \end{bmatrix}, v_{14} = \begin{bmatrix} -0.13 \\ -0.13 \\ -0.13 \\ -0.0 \\ -0.13 \\ 0.42 \\ 0.08 \\ -0.04 \\ -0.04 \\ 0.04 \\ -0.04 \\ 0.05 \\ 0.02 \\ 0.02 \\ 0.02 \\ -0.12 \\ -0.21 \\ 0.77 \\ -0.21 \\ -0.07 \\ -0.21 \end{bmatrix}, v_{15} = \begin{bmatrix} -0.64 \\ 0.75 \\ 0.04 \\ 0.0 \\ -0.16 \\ 0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

$$v_{16} = \begin{bmatrix} 0.39 \\ 0.39 \\ -0.83 \\ -0.0 \\ 0.06 \\ -0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ 0.0 \end{bmatrix}, v_{17} = \begin{bmatrix} -0.44 \\ -0.18 \\ -0.23 \\ -0.0 \\ 0.85 \\ -0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ -0.0 \\ -0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ -0.0 \\ 0.0 \\ 0.0 \end{bmatrix}, v_{18} = \begin{bmatrix} -0.02 \\ -0.02 \\ -0.02 \\ 0.16 \\ -0.02 \\ -0.06 \\ 0.1 \\ -0.13 \\ -0.13 \\ 0.46 \\ -0.13 \\ -0.67 \\ 0.2 \\ 0.2 \\ 0.2 \\ -0.17 \\ -0.06 \\ -0.05 \\ -0.06 \\ 0.31 \\ -0.06 \end{bmatrix}, v_{19} = \begin{bmatrix} -0.07 \\ -0.07 \\ -0.07 \\ 0.14 \\ -0.07 \\ 0.21 \\ -0.02 \\ -0.03 \\ -0.03 \\ 0.16 \\ -0.03 \\ -0.35 \\ -0.06 \\ -0.06 \\ -0.06 \\ 0.63 \\ 0.12 \\ -0.0 \\ 0.12 \\ -0.56 \\ 0.12 \end{bmatrix}$$

$$v_{20} = \begin{bmatrix} 0.2 \\ 0.2 \\ 0.2 \\ -0.69 \\ 0.2 \\ -0.33 \\ 0.3 \\ -0.08 \\ -0.08 \\ 0.11 \\ -0.08 \\ -0.11 \\ -0.0 \\ -0.0 \\ -0.0 \\ 0.13 \\ -0.02 \\ 0.26 \\ -0.02 \\ -0.16 \\ -0.02 \end{bmatrix}, v_{21} = \begin{bmatrix} -0.04 \\ -0.04 \\ -0.04 \\ -0.5 \\ -0.04 \\ 0.72 \\ 0.18 \\ -0.04 \\ -0.04 \\ 0.03 \\ -0.04 \\ -0.0 \\ 0.01 \\ 0.01 \\ 0.01 \\ -0.07 \\ 0.05 \\ -0.4 \\ 0.05 \\ 0.13 \\ 0.05 \end{bmatrix}$$

А вот собственные значения:

$$\lambda_1 = 0,\ \lambda_2 = 0.26,\ \lambda_3 = 0.3,\ \lambda_4 = 0.61,\ \lambda_5 = 3.7$$

$$\lambda_6 = \lambda_7 = \lambda_8 = \lambda_9 = \lambda_{10} = \lambda_{11} = \lambda_{12} = 5,\ \lambda_{13} = 5.13,\ \lambda_{14} = 5.36$$

$$\lambda_{15} = \lambda_{16} = 6 = \lambda_{17} = 6,\ \lambda_{18} = 6.31,\ \lambda_{19} = 6.74,\ \lambda_{20} = 7.11,\ \lambda_{21} = 7.48$$

1.3 Кластеризация при $k = 4$

Здесь k - количество кластеров, на которые мы хотим разбить наш граф. Начнём с оптимального значения (так как граф специально был разбит на 4 группы), чтобы было от чего отталкиваться.

Итак, возьмём k самых малых собственных чисел. Для нас это $\lambda_1 = 0$, $\lambda_2 = 0.26$, $\lambda_3 = 0.3$, $\lambda_4 = 0.61$,. Составим из соответствующих их векторов матрицу

$$V = \begin{bmatrix} | & | & | & | \\ v_1 & v_2 & v_3 & v_4 \\ | & | & | & | \end{bmatrix}$$

Получим:

$$V = \begin{bmatrix} 0.22 & -0.31 & 0.0 & -0.2 \\ 0.22 & -0.31 & 0.0 & -0.2 \\ 0.22 & -0.31 & 0.0 & -0.2 \\ 0.22 & -0.27 & 0.04 & -0.14 \\ 0.22 & -0.31 & 0.0 & -0.2 \\ 0.22 & -0.26 & -0.04 & -0.1 \\ 0.22 & -0.05 & 0.28 & 0.19 \\ 0.22 & 0.03 & 0.33 & 0.26 \\ 0.22 & 0.03 & 0.33 & 0.26 \\ 0.22 & 0.1 & 0.28 & 0.17 \\ 0.22 & 0.03 & 0.33 & 0.26 \\ 0.22 & 0.28 & 0.05 & -0.19 \\ 0.22 & 0.32 & -0.0 & -0.27 \\ 0.22 & 0.32 & -0.0 & -0.27 \\ 0.22 & 0.32 & -0.0 & -0.27 \\ 0.22 & 0.28 & -0.05 & -0.19 \\ 0.22 & 0.02 & -0.33 & 0.26 \\ 0.22 & -0.03 & -0.28 & 0.18 \\ 0.22 & 0.02 & -0.33 & 0.26 \\ 0.22 & 0.07 & -0.28 & 0.17 \\ 0.22 & 0.02 & -0.33 & 0.26 \end{bmatrix}$$

Далее, используя простой код в среде matlab, найдём разбиение на кластеры:

```
clusters = kmeans(V,k); % k = 4
```

Теперь в переменной *clusters* хранятся числа от 1 до k. Вершины попали в один и тот же кластер, если i число совпадает с j. Раскрасим наш граф, исходя из результата работы нашей программы:

```
2
2
2
2
2
2
3
3
3
3
3
1
1
1
1
1
4
4
4
4
4
```

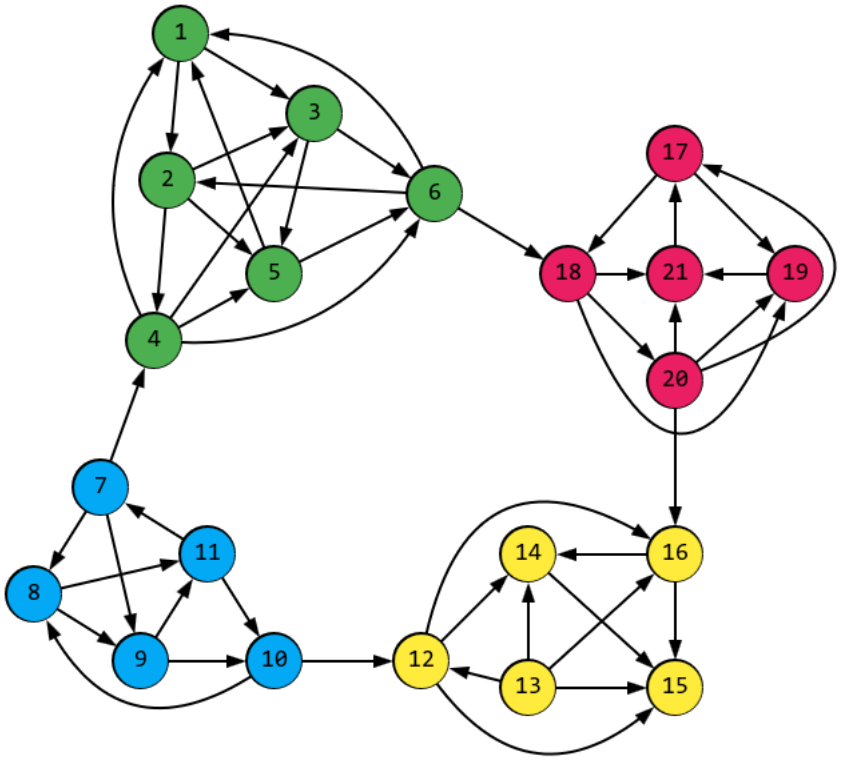


Рис. 2: Кластеризация для k=4

Как видим, программа смогла очень точно определить нужные кластеры. Это успех!

1.4 Кластеризация для $k = 2$

Посмотрим теперь на кластеризацию при различных значениях k.

Пусть сначала $k = 2$. Собственные значения системы будут следующими: $\lambda_1 = 0$, $\lambda_2 = 0.26$. Рассматриваемая же матрица, составленная из отвечающих этим числам векторов, примет вид:

$$V = \begin{bmatrix} 0.22 & -0.31 \\ 0.22 & -0.31 \\ 0.22 & -0.31 \\ 0.22 & -0.27 \\ 0.22 & -0.31 \\ 0.22 & -0.26 \\ 0.22 & -0.05 \\ 0.22 & 0.03 \\ 0.22 & 0.03 \\ 0.22 & 0.1 \\ 0.22 & 0.03 \\ 0.22 & 0.28 \\ 0.22 & 0.32 \\ 0.22 & 0.32 \\ 0.22 & 0.32 \\ 0.22 & 0.28 \\ 0.22 & 0.02 \\ 0.22 & -0.03 \\ 0.22 & 0.02 \\ 0.22 & 0.07 \\ 0.22 & 0.02 \end{bmatrix}$$

Результатом работы программы будет:

```
2
2
2
2
2
2
2
2
2
2
2
2
2
2
2
2
1
1
1
1
1
```

Раскраска графа представлена на рисунке 3.

1.5 Кластеризация для $k = 6$

Первые k минимальные числа Лапласиана L будут следующими: $\lambda_1 = 0,\ \lambda_2 = 0.26,\ \lambda_3 = 0.3,\ \lambda_4 = 0.61,\ \lambda_5 = 3.7$

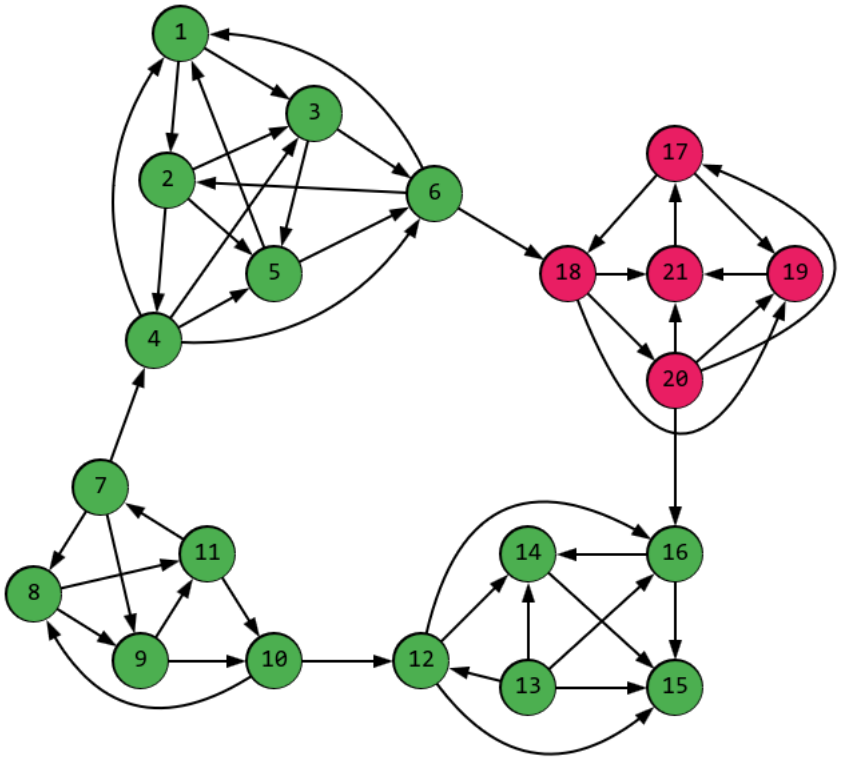


Рис. 3: Кластеризация для k=2

Матрица, составленная из соответственных собственных векторов, в этом случае будет равна

$$V = \begin{bmatrix}
0.22 & -0.31 & 0.0 & -0.2 & -0.06 & 0.0 \\
0.22 & -0.31 & 0.0 & -0.2 & -0.06 & 0.0 \\
0.22 & -0.31 & 0.0 & -0.2 & -0.06 & 0.0 \\
0.22 & -0.27 & 0.04 & -0.14 & 0.15 & -0.0 \\
0.22 & -0.31 & 0.0 & -0.2 & -0.06 & 0.0 \\
0.22 & -0.26 & -0.04 & -0.1 & -0.05 & 0.0 \\
0.22 & -0.05 & 0.28 & 0.19 & 0.65 & -0.0 \\
0.22 & 0.03 & 0.33 & 0.26 & 0.01 & 0.08 \\
0.22 & 0.03 & 0.33 & 0.26 & 0.01 & 0.18 \\
0.22 & 0.1 & 0.28 & 0.17 & -0.67 & -0.0 \\
0.22 & 0.03 & 0.33 & 0.26 & 0.01 & -0.27 \\
0.22 & 0.28 & 0.05 & -0.19 & -0.24 & -0.0 \\
0.22 & 0.32 & -0.0 & -0.27 & 0.1 & 0.64 \\
0.22 & 0.32 & -0.0 & -0.27 & 0.1 & -0.65 \\
0.22 & 0.32 & -0.0 & -0.27 & 0.1 & -0.12 \\
0.22 & 0.28 & -0.05 & -0.19 & 0.07 & 0.13 \\
0.22 & 0.02 & -0.33 & 0.26 & -0.0 & -0.14 \\
0.22 & -0.03 & -0.28 & 0.18 & -0.0 & 0.0 \\
0.22 & 0.02 & -0.33 & 0.26 & -0.0 & -0.01 \\
0.22 & 0.07 & -0.28 & 0.17 & 0.03 & 0.13 \\
0.22 & 0.02 & -0.33 & 0.26 & -0.0 & 0.01
\end{bmatrix}$$

А вот результат работы кода:

```
6
6
6
1
6
6
4
4
4
4
4
3
5
5
5
3
2
2
2
2
2
```

Раскрашенный граф представлен на рисунке 4.

Видим, что как при уменьшении, так и при увеличении k теряется точность (при больших k, например, появляются отдельные островки одиночек, которые, как мы подсознательно понимаем, никакого отношения ни к одному из кластеров не имеют; при маленьких же происходит неуместное объединение в объёмные группы, что нам тоже не нравится).

1.6 Раскрытие магии

Давайте детально разберём метод k-means и поймём, что на самом деле за ним прячется довольно-таки простая математика.

Итак, метод k-means основан на минимизации суммы квадратов расстояний элементов центроидов (неких центров масс). Сначала берутся k произвольных точек в пространстве - мы-то их и называем центроидами (вообще говоря, лучше брать либо сами рассматриваемые точки, либо использовать методы по оптимальному их расположению, ведь качество кластеризации в методе k-means напрямую

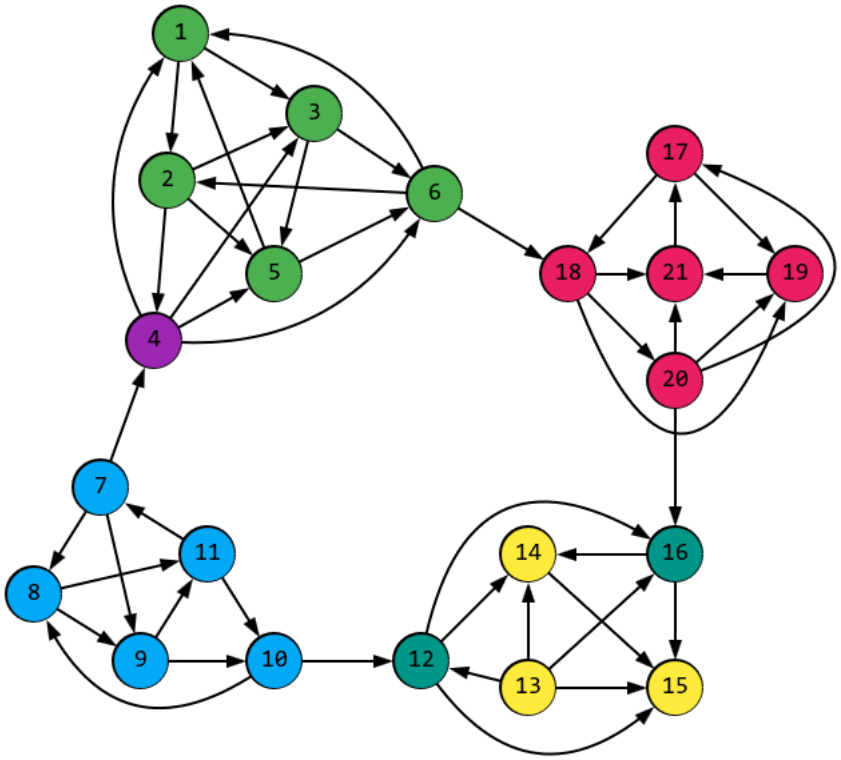


Рис. 4: Кластеризация для k=6

зависит ещё и от того, как мы поставили исходные точки).

Далее каждая точка примыкает к своему центроиду исходя из расстояния до каждого (выбирается минимальное). После для каждой из групп вычисляется новый центр, являющийся средним между значениями всех точек внутри рассматриваемого кластера.

Процедура объединения в кластеры и поиска нового центроида для каждого продолжается до тех пор, пока мы не придём к стабильности (но обычно на число шагов ставится ограничение, оно и понятно - никто не хочет ждать тысячу лет, пока точки разобъёт на кластеры).

Соответственно, качество кластеризации мы можем оценить по сумме расстояний внутри кластеров.

Вот, вообще никакой магии - простая арифметика!

2 Google PageRank алгоритм

В задании попробуем применить алгоритм сортировки страниц, когда-то используемый компанией Google.

2.1 Выбор графа

Перед тем, как приступить к работе, необходимо выбрать ориентированный граф, каждая вершина которого по своей сути будет являться веб-страницей, а стрелка - ссылкой, позволяющей пользователю перейти с одной страницы на другую.

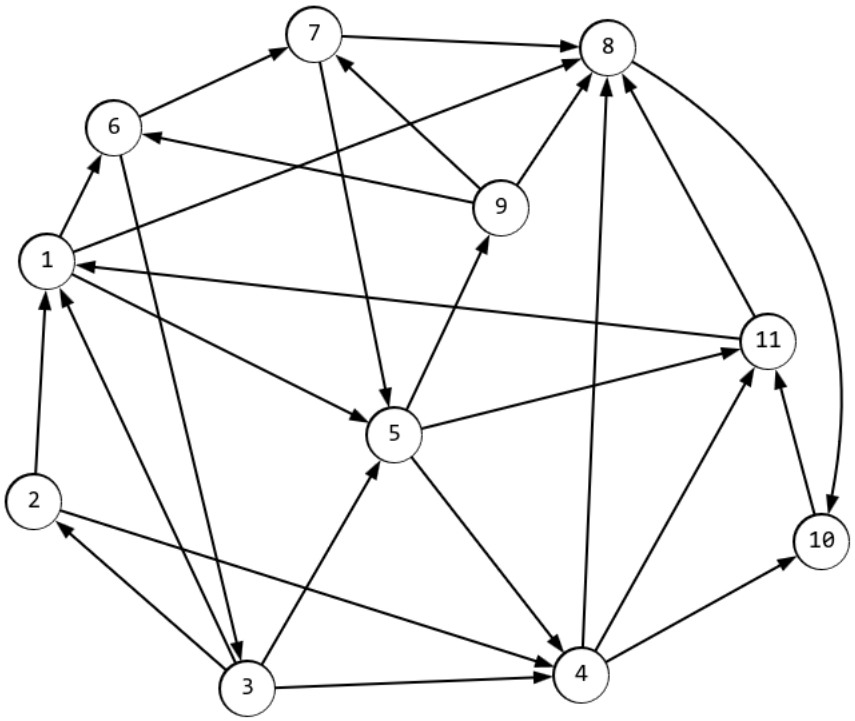


Рис. 5: Ориентированный граф

У составленного графа 11 вершин и 26 ребёр, соединяющих их.

2.2 Составление матрицы и её анализ

Составим теперь матрицу следующего вида:

$$M = \begin{bmatrix} m_{11} & m_{12} & \cdots & m_{1n} \\ m_{21} & m_{22} & \cdots & m_{2n} \\ m_{11} & m_{12} & \cdots & m_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & m_{nn} \end{bmatrix},$$

где $m_{ij} = \frac{\text{число стрелок, выходящих из } j\text{-й вершины и входящих в } i\text{-ую вершину}}{\text{общее число стрелок, выходящих из } j\text{-ой вершины}}$

Итак, получаем следующую матрицу:

$$M = \begin{bmatrix} 0 & 0.5 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.5 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.5 & 0.25 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.25 & 0 & 0 & 0 & 0.5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.33 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.5 & 0 & 0 & 0.33 & 0 & 0 \\ 1 & 0 & 0 & 0.33 & 0 & 0 & 0.5 & 0 & 0.33 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.33 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.33 & 0.33 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Произведем сингулярное разложение и найдём собственный вектор, соответствующий наибольшему собственному числу:

$$v_{max} = \begin{bmatrix} 0.28 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0.55 \\ 0 \\ 0.55 \\ 0.55 \end{bmatrix}, \lambda_{max} = 1$$

2.3 Ранжировка

Переключимся с Matlab на Python (потому что все примеры в статьях с pagerank приведены на Python, а ещё потому что мне не

хочется активно пользоваться функциями в Matlab'е - они там странные :)). Используем следующий код для провередния ранжировки страниц:

```
import numpy as np

def pagerank(A, k):
    n = A.shape[1]
    v = np.ones(n) / n
    for i in range(k):
        v = A.dot(v)
    return v

A = [[0, 0.5, 0.25, 0,   0,   0,   0,   0,   0,   0, 0.5],
     [0,   0, 0.25, 0,   0,   0,   0,   0,   0,   0,   0],
     [0,   0,    0, 0,   0, 0.5,   0,   0,   0,   0,   0],
     [0, 0.5, 0.25, 0, 1/3,   0,   0,   0,   0,   0,   0],
     [0,   0, 0.25, 0,   0,   0, 0.5,   0,   0,   0,   0],
     [0,   0,    0, 0,   0,   0,   0,   0, 0.33,   0,   0],
     [0,   0,    0, 0,   0, 0.5,   0,   0, 0.33,   0,   0],
     [1,   0,    0, 1/3, 0,   0, 0.5,   0, 0.33,   0, 0.5],
     [0,   0,    0, 0,   1/3, 0,   0,   0,   0,   0,   0],
     [0,   0,    0, 1/3, 0,   0,   0,   1,   0,   0,   0],
     [0,   0,    0, 1/3, 1/3, 0,   0,   0,   0,   1,   0]]
A = np.array(A)
v = pagerank(A, 100)

for i in range(len(v)):
    print(f"{i+1}: {round(v[i], 3)}")
```

Здесь матрица A равна M из пункта выше. Мы не говорили, но по элементы m_{ij}, из которых составлена эта матрица, по своей сути означают вероятность попадания из j страницы в i при учёте, что наш пользователь никак не опирается на прочие факторы страницы i (таких как ссылка и прочее - не думаю, что кто-то захотел перейти на сайт с названием lolka.ru).

Итак, алгоритм pagerank полностью основан на марковских процессах. Изначально мы можем попасть на любую из j страниц с равной вероятностью (её задаёт отнормированный вектор v), после мы k число раз умножаемся на A (марковские процессы в силе) и получаем результирующий вектор вероятности попадания на ту или иную страницу *в целом* - звучит круто!

Вот результат работы для нашей матрицы $A = M$ (были убраны

числа, так как слева и так идёт нумерация):

```
1  0.143
2  0.0
3  0.0
4  0.0
5  0.0
6  0.0
7  0.0
8  0.285
9  0.0
10 0.285
11 0.285
```

Кстати, если вспомнить собственный вектор v_{max}, соответствующий максимальному собственному числу, то он показал те же условно «важные» страницы, причем он также учёл распределение вероятностей - у 8, 10 и 11 они наибольшие и одинаковые, а вот у 1 она чуть поменьше. Оно и понятно, ведь в марковских процессах в итоге берут вверх самые большие!

Стоит также отметить, что в алгоритме мы не учитывали параметр затухания d, который так же играет важную роль, - в реальном мире при переходе от страницы к странице мы основываемся не только на наличии ссылки, но и её сути!

На этом всё, спасибо!